IntersLand HRMS — Master Development Reference

Version 2.0 | March 2026 | Confidential

QUICK REFERENCE: WHO OWNS WHAT

Module	Developer	Branch	Key Files
Administration + Payroll	**Aditya**	`aditya`	`adminController.ts`, `payroll.controller.ts`, `adminRoutes.ts`, `payroll.routes.ts`, `salary.routes.ts`
Performance (PMS)	**Javeriya & Anjali**	`pms`	`performanceController.ts`, `performanceRoutes.ts`, `PerformanceCycle.ts`, `PerformanceReview.ts`
Recruitment & Staffing	**Malhar & Pankaj**	`recruitment`	`candidate.controller.ts`, `requisition.controller.ts`, `job.controller.ts`, `offer.controller.ts`, `application.controller.ts`
Reports & Analytics	**Sreeshanth**	`reports`	`reports.controller.ts`, `reports.routes.ts`, `reports.service.ts`
Employee Center	**Saurav**	`employee-center`	`employee.controller.ts`, `onboardingController.ts`, `transferController.ts`, `Employee.model.ts`
Attendance	**Vishal**	`attendance`	`attendance.controller.ts`, `attendance.routes.ts`, `Attendance.model.ts`, `shift.controller.ts`

> **Shared Ownership (All Devs Must Know)**
>
> - `backend/middleware/auth.middleware.ts` — JWT protect guard. Do NOT modify without Aditya review.
> - `backend/middleware/role.middleware.ts` — RBAC matrix middleware. Do NOT modify without Aditya review.
> - `backend/models/User.model.ts` — Central user model. Any schema change MUST be discussed in standup first.
> - `backend/models/Employee.model.ts` — Central employee model. Saurav owns this; coordinate before changing.
> - `frontend/src/services/apiClient.js` — Shared API wrapper. Read the ⚠ WARNING section below before using.

⚠ CRITICAL: READ BEFORE WRITING ANY CODE

1. Auth: JWT Only — No Firebase for Auth

The project uses **JWT only** for authentication. Firebase is present in the repo (`config/firebase.ts`, `lib/firebase.js`) but is **NOT used for login/auth**. Do not introduce Firebase Auth calls.

- JWT token is stored in `localStorage` as `token`
- All protected API calls must send: `Authorization: Bearer <token>`
- `backend/middleware/auth.middleware.ts` → `protect` middleware validates the JWT and attaches `req.user`
- `backend/middleware/role.middleware.ts` → `authorizeByMatrix` checks DB permissions dynamically

2. ⚠ apiClient Auto-Unwrapping Bug — CRITICAL FOR ALL FRONTEND DEVS

`frontend/src/services/apiClient.js` has an **interceptor that auto-unwraps `response.data`**.

```
// What axios normally returns:
axios.get('/api/employees')
// → { data: { success: true, data: [...] } }

// What apiClient returns AFTER interceptor unwraps:
apiClient.get('/api/employees')
// → { success: true, data: [...] }   ← response.data is already unwrapped
```

The Rule: When using `apiClient`, access `.data` once — NOT twice.

```
// ✅ CORRECT
const res = await apiClient.get('/api/employees');
const employees = res.data;  // res is already response.data

// ❌ WRONG — causes undefined
const res = await apiClient.get('/api/employees');
const employees = res.data.data;  // double-unwrap = undefined
```

Always check existing service files (`adminService.js`, `attendanceService.js`, etc.) to see the pattern used before writing new service functions.

3. Role Name Format in JWT vs DB

JWT token encodes role as uppercase-no-space: `HRMANAGER`, `DEPARTMENTMANAGER`, `DIRECTMANAGER`
DB stores roles as: `HR Manager`, `Department Manager`, `Direct Manager`

`role.middleware.ts` handles normalization. Use the constants from `frontend/src/constants/roles.js`:

```
import { ROLES } from '../constants/roles';
// ROLES.HRMANAGER = 'HR_MANAGER'
```

```
// ROLES.ADMIN = 'ADMIN'
// etc.
```

4. No "Camp Boss" Role

There is **no Camp Boss role** in this system. Any documentation referencing Camp Boss is outdated. The role hierarchy is:

Level	Role	Constant
11	Super Admin / Admin	`ADMIN`
10	CEO	`CEO`
9	Country Manager	`COUNTRYMANAGER`
8	HR Manager	`HRMANAGER`
7	Finance Manager / HR Officer	`HROFFICER`
6	Department Manager	`DEPARTMENTMANAGER`
5	Direct Manager	`DIRECTMANAGER`
4	Payroll Officer	`PAYROLLOFFICER`
3	Project Manager	`PROJECTMANAGER`
1	Employee	`EMPLOYEE`

5. Employee-User Link (Critical for Payroll, Attendance, Reports)

Every `User` document must have an `employeeId` field pointing to its `Employee` document. This link is the backbone of all cross-module lookups.

```
User._id  →  User.employeeId  →  Employee._id
```

- Created via **Manual Add** (Admin creates user → Employee doc auto-created)
- Created via **Bulk Upload** (CSV → `adminController.ts` creates both and links them)
- Created via **Onboarding** (`onboardingController.ts` upserts Employee, then sets `employeeId` on User)

If `user.employeeId` is `null`, payroll, salary, and attendance lookups **will silently fail**. If you suspect broken links, run: `npx ts-node scripts/migrateEmployees.ts`

SYSTEM ARCHITECTURE

Tech Stack

- **Backend:** Node.js + TypeScript + Express.js

- **Frontend:** React 18 + Vite + TailwindCSS + shadcn/ui
- **Database:** MongoDB Atlas (Mongoose ODM)
- **Auth:** JWT (jsonwebtoken) — tokens expire in 7 days
- **File Storage:** Local `uploads/` directory + Cloudinary (for Digital Locker documents)
- **PDF Generation:** `backend/utils/payslip.generator.ts`
- **Email:** SMTP via nodemailer (configured in Settings)

Ports

- Backend: `http://localhost:5000`
- Frontend: `http://localhost:5173`

Environment Variables (backend `.env`)

```
MONGODB_URI=
JWT_SECRET=
PORT=5000
CLOUDINARY_CLOUD_NAME=
CLOUDINARY_API_KEY=
CLOUDINARY_API_SECRET=
```

Directory Structure (Key Files Only)

```
HRMS/
├── backend/
│   ├── controllers/          ← Business logic per module
│   ├── middleware/
│   │   ├── auth.middleware.ts     ← JWT protect()
│   │   └── role.middleware.ts     ← authorizeByMatrix(), authorize()
│   ├── models/               ← Mongoose schemas
│   ├── routes/               ← Express routers
│   ├── services/
│   │   └── reports.service.ts
│   ├── utils/
│   │   ├── payslip.generator.ts
│   │   ├── responseHandler.ts    ← sendSuccess(), sendError()
│   │   └── roleSeeder.ts         ← Seeds default roles on startup
│   ├── scripts/
│   │   ├── migrateEmployees.ts   ← Backfill employeeId on Users
│   │   └── seedTestData.ts       ← Seed permissions for testing
│   └── server.ts             ← All routes registered here
├── frontend/src/
│   ├── services/
│   │   └── apiClient.js          ← ⚠ Auto-unwraps response.data
│   ├── contexts/             ← AuthContext, UserContext, RoleContext
│   ├── constants/roles.js    ← ROLES enum — use this everywhere
│   └── pages/modules/        ← One folder per module
```

RBAC Matrix Permission Keys

Each role in MongoDB has a `permissions` map. Keys used across the system:

Module	permissionKey
Dashboard	`dashboard`
Employee Center	`employees`
Recruitment	`recruitment`
Time & Attendance	`attendance`
Payroll	`payroll`
Performance	`performance`
Reports	`reports`
Administration	`settings`, `roles`, `audit`

Permission shape per key: `{ read: boolean, write: boolean, delete: boolean }`

Backend check:

```
import { authorizeByMatrix } from '../middleware/role.middleware';
router.get('/endpoint', protect, authorizeByMatrix('payroll', 'read'),
controller);
```

Frontend check:

```
import { useUser } from '../contexts/UserContext';
const { permissions } = useUser();
const canWrite = permissions?.payroll?.write === true;
```

MODULE 1: ADMINISTRATION

Owner: Aditya | Files: `adminController.ts`, `adminRoutes.ts`, `roleController.ts`, `settingsController.ts`, `audit.controller.ts`, `departmentController.ts`

Roles With Access

Role	User Mgmt	Role Perms	Company Structure	Audit Logs	System Settings	API Integrations
Super Admin / Admin	Full Control	Full Control	Full Control	Full Access	Full Control	Full Control

Role	User Mgmt	Role Perms	Company Structure	Audit Logs	System Settings	API Integrations
HR Manager	View Only	Assign Roles	Full Control	View Only (HR Only)	HR Settings Only	No
IT Administrator	View Only	No	No	System Logs Only	Technical Only	Full Control
CEO	View Only	No	View Only	Executive Summary	No	No
All Others	No	No	No	No	No	No

Key API Endpoints

```
POST   /api/admin/users                → Create user (Admin)
GET    /api/admin/users                → List all users (Admin)
PUT    /api/admin/users/:id            → Update user (Admin)
DELETE /api/admin/users/:id            → Soft delete (Admin)
POST   /api/admin/users/:id/deactivate
POST   /api/admin/users/bulk-upload    → CSV bulk import
POST   /api/admin/users/:userId/assign-role
GET    /api/admin/roles                → List roles
POST   /api/admin/roles                → Create custom role
PUT    /api/admin/roles/:id            → Update role permissions
GET    /api/admin/audit-logs           → All logs
POST   /api/admin/audit-logs/export    → Export CSV
GET    /api/admin/settings             → All settings
PUT    /api/admin/settings/:key        → Update setting
GET    /api/admin/company-structure    → Org tree
POST   /api/admin/departments          → Create department (HR Manager)
```

Key Business Rules

- Admin cannot change their own role (self-demotion locked)
- System requires minimum 1 Admin at all times
- CEO role is unique — only 1 user can hold it
- Bulk uploads validated row-by-row; partial success allowed
- All destructive actions logged to AuditLog with before/after values
- Users limited to 3 concurrent sessions; 4th login invalidates oldest

Inter-Module Dependencies

- On user creation → triggers Employee doc creation (`Employee.model.ts`)
- On department create/update → triggers `departmentSync` to propagate to Attendance rosters, Payroll cost centers, Performance hierarchy
- On role change → immediately updates `req.user` permissions on next JWT refresh

---

MODULE 2: PAYROLL & COMPENSATION

Owner: Aditya | Files: `payroll.controller.ts`, `payroll.routes.ts`, `salary.routes.ts`, `payroll.model.ts`, `salaryStructure.model.ts`, `indemnity.model.ts`, `ticket.model.ts`

Roles With Access

Role	Define Salary	Process Payroll	Approve Payroll	Disburse	Advances	Analytics
HR Manager	✅ Full Control	Initiate Only	✅ Yes	No	✅ Full	✅ Full
Payroll Officer	No	✅ Execute	No	✅ Trigger	Process	Processing Logs
Finance Manager	Review Only	No	✅ Verify	✅ Approve	Approve Loans	Financial Reports
CEO	No	No	✅ Override (>10L)	No	Approve >50K	Executive KPIs
Admin	No	No	No	No	No	System Logs only
Department Manager	No	No	No	No	View/Approve dept	Dept Cost Only
Employee	No	No	No	No	Request Own	Own Payslips

Salary Structure (Current Schema — `salaryStructure.model.ts`)

```
{
  userId: ObjectId,       // links to User
  employeeId: string,     // kept for backward compat
  basic: number,
  hra: number,
  transport: number,
  medical: number,
  otherAllowance: number,
  pf: number,
  tax: number,
  insurance: number,
  effectiveFrom: Date
}
```

> **Note:** Salary data also lives as an embedded sub-document in `Employee.model.ts` under `employee.salary`. The salary routes (`salary.routes.ts`) read/write from Employee.salary. The `payroll.controller.ts` must always look up salary via `user.employeeId → employee.salary` first, then fallback to `salaryStructure` collection.

Monthly Payroll Cycle

```
26th  → Attendance data auto-imported from Attendance module
(islocked=true on attendance)
27th  → Payroll Officer calculates LOP (unapproved leaves from Leave
module)
27th  → Payroll Officer deducts active advance EMIs (FIFO order)
28th  → Payroll Officer adds approved reimbursements
28th  → Statutory calculations: PF(12%), ESI(0.75% emp/3.25% employer),
PT(₹200 Maharashtra), TDS
28th  → System computes gross-to-net
29th  → Finance Manager verifies
30th  → HR Manager approves
30th  → CEO authorizes if total > ₹10L
1st   → Payroll Officer generates PDF payslips
1-2nd → Finance Manager triggers bank transfer
3rd   → HR Manager locks cycle (islocked=true — IMMUTABLE after this)
```

Key API Endpoints

```
GET    /api/salary/:userId           → Get employee salary (HR roles)
PUT    /api/salary/:userId           → Update salary structure (HR
Manager)
GET    /api/salary/:userId/self      → Employee views own salary
POST   /api/payroll/run              → Initiate payroll run
POST   /api/payroll/calculate        → Calculate gross-to-net
GET    /api/payroll/status/:runId    → Check status
POST   /api/payroll/:runId/approve   → HR Manager approval
POST   /api/payroll/:runId/lock      → Lock cycle
POST   /api/payroll/:runId/disburse  → Finance Manager triggers transfer
POST   /api/payslip/generate/:runId  → Bulk payslip generation
GET    /api/payslip/:employeeId/:month/:year → Download payslip
POST   /api/advance/request          → Employee advance request
POST   /api/advance/:id/approve      → Role-based approval
POST   /api/reimbursement/submit     → Submit claim
POST   /api/indemnity/calculate/:id  → Calculate indemnity
```

Statutory Rates

- **PF:** 12% employee + 12% employer (3.67% PF + 8.33% Pension) — applicable for basic ≤ ₹15,000
- **ESI:** 0.75% employee + 3.25% employer — applicable for gross ≤ ₹21,000
- **PT (Maharashtra):** ₹200/month, annual cap ₹2,500
- **TDS:** Per IT slab after 80C deductions; Form 16 by May 31st

Payroll Lock Policy (Revised — addressing Strict Lock Con)

- **Attendance Correction Window:** Managers can correct attendance until **20th of month**
- **Payroll Preview Window:** HR Manager can review/edit salary inputs until **29th**

- **Hard Lock:** HR Manager locks on **3rd of next month** — after this, only Admin emergency unlock (with mandatory audit trail + reason)
- Payroll Officer can request a "Grace Unlock" for specific employee records (not full payroll) until the 2nd, requires HR Manager approval

Inter-Module Dependencies

```
← Attendance Module    : import attendance records, calculate LOP days
← Leave Module         : import unapproved leave count for LOP formula
← Employee Center      : validate active employees, get
designation/department
← Performance Module   : fetch performance rating for bonus calculation
← Recruitment/Onboarding: auto-create salary structure when candidate
accepts offer
```

MODULE 3: EMPLOYEE CENTER

Owner: Saurav | Files: `employee.controller.ts`, `onboardingController.ts`, `transferController.ts`, `Employee.model.ts`, `Transfer.model.ts`, `Document.model.ts`

Sub-Modules

1. **All Employees** — Read-only global directory (`AllEmployees.jsx`)
2. **Onboarding KYC** — 4-stage pipeline: Personal → Employment → Bank/Statutory → Documents (`Onboarding.jsx`)
3. **Transfers & Promotions** — Employee initiates, Admin approves (`TransferPage.jsx`)
4. **Digital Locker** — Cloudinary-backed document vault (`EmployeeProfileView.jsx`)

Roles With Access

Role	View Directory	Execute Onboarding	Manage Transfers	Digital Locker
Admin	Full Org	Full Control	Approve/Reject	Full Control
HR Manager	Full Org	Support/Verify	Recommend	Upload/Verify
CEO	Full Org	No	Final Override	Read Only
Employee	Self Only	No	Initiate Own	Upload Self Only

Employee ID Generation

- Format: `EMP-XXXX` (e.g., `EMP-0001`)
- Auto-generated during Onboarding KYC stage completion
- This `employeeId` (stored on `User.employeeId`) is the **primary cross-module key** used by Payroll, Attendance, and Performance

Key API Endpoints

```
GET    /api/employees              → All employees (Admin/HR)
GET    /api/employees/:id          → Employee profile
POST   /api/onboarding             → Save onboarding progress
GET    /api/onboarding/:email      → Get onboarding details
GET    /api/onboarding/all         → All onboarding records
POST   /api/transfers              → Initiate transfer (Employee)
PUT    /api/transfers/:id/approve  → Approve transfer (Admin)
GET    /api/documents/:employeeId  → Get documents
POST   /api/documents/upload       → Upload document (Cloudinary)
```

Hybrid Storage Architecture

- **MongoDB:** Stores all text data (personal info, employment details, bank details)
- **Cloudinary:** All physical files (ID proofs, certificates, photos) dynamically routed by category
- **Local `uploads/onboarding/`:** Temporary staging before Cloudinary push

Inter-Module Dependencies

```
→ Payroll Module        : Provides employeeId, salary package from offer
letter
→ Attendance Module     : Provides employeeId, department, shift
allocation on onboarding complete
→ Performance Module    : Provides employeeId, manager assignment, role
for appraisal cycle
→ Recruitment Module    : Receives candidate data when offer is accepted
(trigger: createEmployeeRecord)
→ Administration Module : User creation in Admin triggers Employee doc
creation here
```

MODULE 4: RECRUITMENT & STAFFING

Owner: Malhar & Pankaj | Files: `candidate.controller.ts`, `requisition.controller.ts`, `job.controller.ts`, `offer.controller.ts`, `application.controller.ts`

Roles With Access

Role	View	Create Requisitions	Approve	Modify Data
HR Manager	Full Org	✅ Yes	✅ Yes (Final)	✅ Yes
HR Officer	Department	✅ Yes	No	✅ Yes
Admin	Full Org	No	No	Read Only
CEO	Full Org	No	Final Override	Read Only
Employee	None	No	No	No

Recruitment Lifecycle (Revised — Parallel Processing Supported)

The pipeline supports **parallel tracks** for some stages to remove bottlenecks:

```
Stage 1:  HR Officer creates Job Requisition
Stage 2:  HR Manager approves Requisition  [APPROVAL GATE]
Stage 3:  Job posting published (job.controller.ts → job.model.ts)
Stage 4:  Candidates submit applications (public: /careers, /apply-job
pages)
Stage 5:  HR Officer screens resumes
Stage 6:  Interviews scheduled (ScheduleInterviewModal.jsx)
Stage 7:  Interview feedback recorded
          ↓ PARALLEL TRACK (can run alongside Stage 7)
          Stage 7a: Background check initiated
          Stage 7b: Document collection started (offer documents)
Stage 8:  HR Manager approves candidate  [APPROVAL GATE]
Stage 9:  Offer letter generated (offer.controller.ts → PDF)
Stage 10: Candidate accepts offer
Stage 11: System auto-creates Employee record → triggers Onboarding
```

> ⚠ Stages 2 and 8 remain mandatory approval gates and cannot be skipped. If HR Manager is unavailable >24 hours, escalates to Admin.

Key API Endpoints

```
POST   /api/requisitions              → Create job requisition (HR
Officer)
PUT    /api/requisitions/:id/approve  → Approve requisition (HR Manager)
GET    /api/job-openings              → List all job openings
POST   /api/candidates                → Add candidate
GET    /api/candidates                → List candidates
PUT    /api/candidates/:id            → Update candidate status
POST   /api/applications              → Submit application (public)
GET    /api/applications              → List applications
POST   /api/offers/generate           → Generate offer letter PDF
PUT    /api/offers/:id/accept         → Candidate accepts offer
```

Database Models

- `Requisition.model.ts` — Job requisition with approval status
- `Candidate.model.ts` — Candidate profile with unique CandidateID
- `application.model.ts` — Links candidate to job requisition
- `job.model.ts` — Job posting details
- `Offer.model.ts` — Offer letter terms and acceptance status

Inter-Module Dependencies

```
→ Employee Center       : On offer acceptance, POST to /api/onboarding
with candidate data
→ Payroll Module        : Pass offered salary package to create initial
salary structure
→ Attendance Module     : Pass department + shift allocation from job role
→ Performance Module    : Pass manager assignment for probation review
cycle
```

Public-Facing Pages (No Auth Required)

- `/careers` → `Careers.jsx` — Lists open positions
- `/apply-job` → `ApplyJob.jsx` — Application form
- `/application-status` → `ApplicationStatus.jsx` — Candidate tracks own application

MODULE 5: TIME & ATTENDANCE

Owner: Vishal | Files: `attendance.controller.ts`, `attendance.routes.ts`, `Attendance.model.ts`, `shift.controller.ts`, `holiday.controller.ts`

Roles With Access

Role	View Scope	Clock In/Out	Manual Correction	Approve OT	Shift Config
Admin	Organization	No	Yes	No	Yes
HR Manager	Organization	Yes	Yes	Yes	Yes
HR Officer	Organization	Yes	Support	Yes	Support
Dept Manager	Department	Yes	Dept Only	Yes (Dept)	Cost Center
Direct Manager	Team	Yes	Team Only	Yes (Team)	No
Payroll Officer	Organization (Read)	No	No	No	No
Project Manager	Project Site	Yes	No	Limited	No
Employee	Self Only	Yes	Request Only	No	No

Monthly Processing Calendar

```
Daily     → Direct/Dept Manager validates team attendance
By 20th   → All managers must approve/reject attendance (auto-escalates to
HR Manager if missed)
20th      → ⚠ ATTENDANCE CORRECTION WINDOW CLOSES (Payroll reads from this
point)
```

```
22nd      → HR Officer checks for unapproved/missing logs
25th      → Payroll Officer initiates payroll run (attendance data frozen)
```

Escalation Rules

- If Direct Manager (Level 5) absent → auto-route to Dept Manager (Level 6)
- If Dept Manager also absent → auto-route to HR Manager (Level 8)
- Ensures payroll is never delayed

Key API Endpoints

```
POST   /api/attendance/clock-in        → Employee clock in
POST   /api/attendance/clock-out       → Employee clock out
GET    /api/attendance                 → List records (scoped by role)
PUT    /api/attendance/:id/correct     → Manual correction
PUT    /api/attendance/:id/approve     → Manager approval
POST   /api/attendance/lock            → Payroll Officer locks month
GET    /api/attendance/report          → Attendance report
GET    /api/shifts                     → List shifts
POST   /api/shifts                     → Create shift (HR Manager)
POST   /api/shifts/:id/assign          → Assign shift to employee
GET    /api/holidays                   → List holidays
POST   /api/holidays                   → Create holiday (HR Manager)
```

Payroll Lock Enforcement

Once `isLocked: true` is set on attendance records for a period:

- `PUT /api/attendance/:id/correct` returns **403 Forbidden** for all roles except Admin (emergency)
- Only Admin Level 11 can unlock in emergency (creates mandatory audit log entry)

Inter-Module Dependencies

```
→ Payroll Module    : Exports attendance data on the 25th (present days,
absent days, OT hours, LOP days)
← Administration    : Receives team roster updates when department/manager
changes
← Employee Center   : Receives new employee shift allocation on onboarding
complete
→ Reports Module    : Provides raw attendance data for analytics
dashboards
```

MODULE 6: PERFORMANCE MANAGEMENT (PMS)

Owner: Javeriya & Anjali | Files: `performanceController.ts`, `performanceRoutes.ts`, `PerformanceCycle.ts`, `PerformanceReview.ts`

Roles With Access

Role	View Scope	Create Cycle	Assign Review	Submit Review	Approve	Analytics
HR Manager	Organization	✅ Yes	✅ All	✅ Yes	✅ Yes	Full
CEO	Executive Only	No	No	No	✅ Executive	Executive KPIs
HR Officer	Organization	No	Limited Support	✅ Yes	No	Read Only
Dept Manager	Department	No	Own Dept	✅ Yes	✅ Own Dept	Dept Only
Direct Manager	Team	No	Own Team	✅ Yes	✅ Own Team	Team Only
Employee	Self Only	No	No	Self-Assessment Only	No	Own Only
Admin	**None**	No	No	No	No	No

> ⚠ Admin has **zero access** to PMS. Performance is managed entirely by HR.

Review Cycle Stages (Revised — Streamlined for Small Teams)

```
Stage 1: HR Manager creates cycle (name, dates, participants)
Stage 2: Goal setting — HR Manager sets org goals; employees set personal goals (Week 1–2)
Stage 3: Employee self-assessment (mandatory before manager can submit review)
Stage 4: Direct/Dept Manager submits review
Stage 5: HR Manager validates, flags anomalies
Stage 6: CEO reviews executive-level staff only
Stage 7: Calibration session (OPTIONAL — recommended for orgs >50 employees)
Stage 8: HR Manager approves and publishes results
Stage 9: Feedback issued; PIP initiated if needed (HR Manager only)
```

> **Calibration sessions are optional** for this project scope. HR Manager can skip Stage 7 and go directly to Stage 8.

Key API Endpoints

```
POST   /api/performance/cycles              → Create cycle (HR Manager)
GET    /api/performance/cycles              → List cycles (scoped by role)
POST   /api/performance/cycles/:id/assign   → Assign reviewers
POST   /api/performance/reviews             → Submit review
PUT    /api/performance/reviews/:id/approve → Approve review
GET    /api/performance/analytics           → Analytics (role-scoped)
GET    /api/performance/cycles/:id/details  → Cycle details
```

Frontend Pages

```
frontend/src/pages/modules/performance/
├── AdminPerformance.jsx      ← HR Manager full cycle management
├── PerformanceAnalytics.jsx  ← Analytics dashboard
├── ReviewDetailPage.jsx      ← Individual review details
└── ViewCycleDetails.jsx      ← Cycle overview

frontend/src/pages/role/
├── HR_MANAGER/Appraisal.jsx
├── DEPARTMENT_MANAGER/Appraisal.jsx
├── DIRECT_MANAGER/Appraisal.jsx
└── EMPLOYEE/Appraisal.jsx
```

Inter-Module Dependencies

```
← Employee Center   : Receives employeeId, manager assignment, role when
employee is onboarded
→ Payroll Module    : Provides performance rating for quarterly/annual
bonus calculation
← Administration    : Receives hierarchy updates when manager changes (re-
routes review assignments)
→ Reports Module    : Provides performance scores for analytics dashboards
```

MODULE 7: REPORTS & ANALYTICS

Owner: Sreeshanth | Files: `reports.controller.ts`, `reports.routes.ts`, `reports.service.ts`

Roles With Access

Role	Dashboard	Export	Advanced Analytics
CEO	Full Org	✅ Yes	Executive KPIs
HR Manager	Full Org	✅ Yes	Full
HR Officer	Full Org	Limited	Partial

Role	Dashboard	Export	Advanced Analytics
Dept Manager	Department	No	Dept Only
Direct Manager	Team	No	Team Only
Employee	Self Only	No	Own productivity only
Admin	**None**	No	No access
Super Admin	Full Org	✅ Yes	System Override

Dashboard Sections

1. **Executive Overview** — Workforce Health Score, KPI Summary, AI Insights, Budget Overview
2. **Workforce Analytics** — Trend charts, attendance patterns, department distribution, leave breakdown
3. **Productivity Intelligence** — Real-time status, heatmaps, app usage categories, idle detection, work-life balance
4. **Performance Analytics** — Score distribution, compliance metrics, PIP tracking

Dynamic Filters

All charts and tables update when these filters change:

- Period (Last 3M / 6M / Yearly / Custom)
- Start Date / End Date
- Department
- Role

Export Formats

- **CSV** — Structured data export
- **PDF** — Formatted analytical report
- **Excel** — Planned (future enhancement)

Key API Endpoints

```
GET    /api/reports/dashboard          → All dashboard data (role-scoped)
GET    /api/reports/workforce          → Workforce analytics
GET    /api/reports/attendance         → Attendance analytics
GET    /api/reports/performance        → Performance analytics
GET    /api/reports/productivity       → Productivity metrics
POST   /api/reports/export             → Export report (CSV/PDF)
```

Frontend Components

```
frontend/src/components/reports/
├── KPISection.jsx             ← Top KPI cards
```

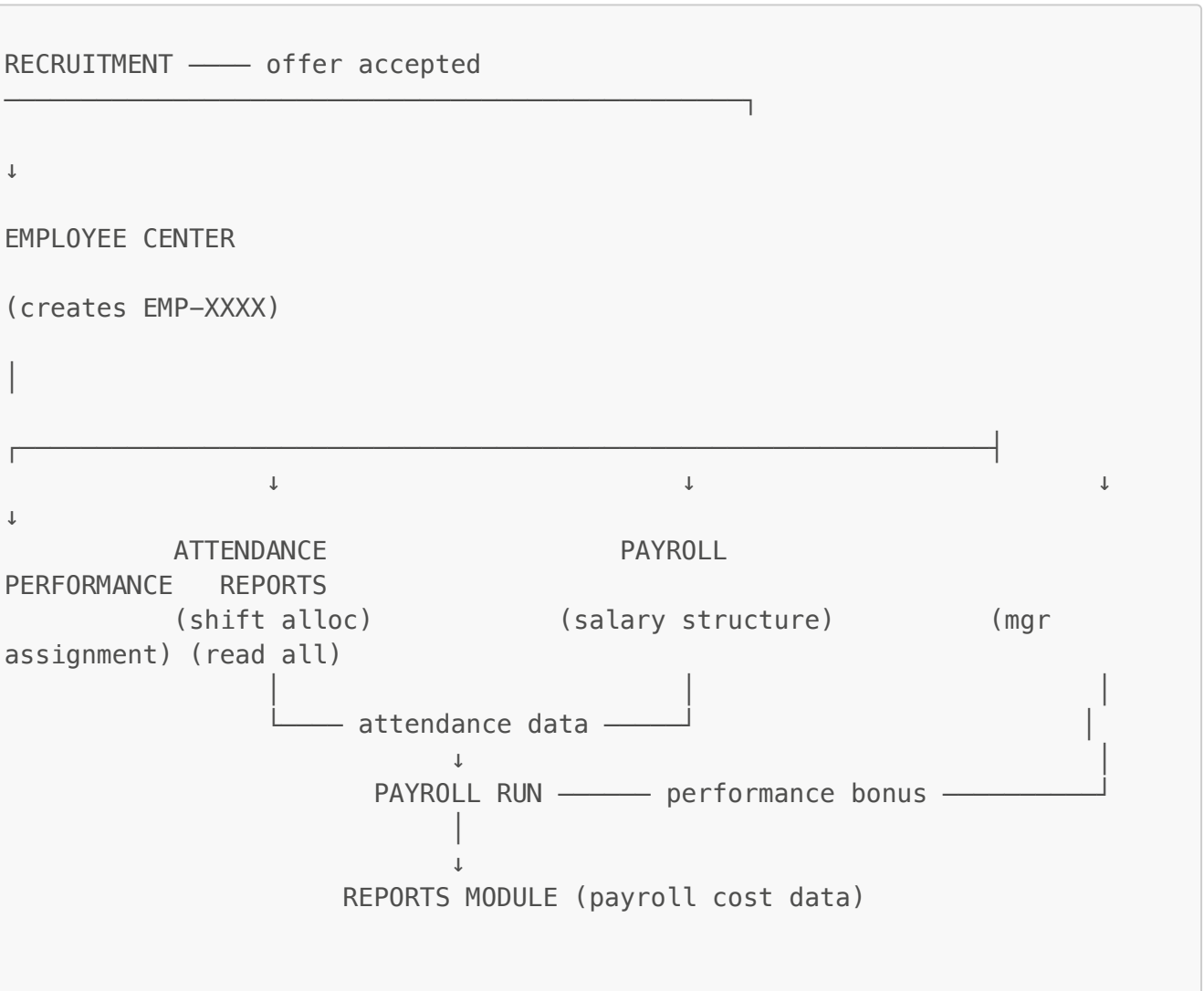
```
├── WorkforceCharts.jsx        ← Trend charts
├── AttendanceCharts.jsx       ← Attendance breakdown
├── PerformanceSection.jsx     ← Performance insights
├── ExecutiveInsights.jsx      ← CEO-level view
├── DepartmentWorkingHours.jsx ← Hours per dept
├── FiltersSection.jsx         ← All filter controls
└── WorkforceHealthMeter.jsx   ← Health score widget
```

Inter-Module Dependencies (Read-Only Consumer)

```
← Employee Center   : Employee headcount, department distribution
← Attendance        : Attendance records, OT hours, absenteeism
← Payroll           : Payroll cost, salary expense per department
← Performance       : Appraisal scores, PIP counts
← Leave             : Leave patterns, carry-forward data
```

> Reports module is **read-only**. It never writes to other modules' collections. Always query via `reports.service.ts`.

CROSS-MODULE DATA FLOW MAP



```
ADMINISTRATION
     │
     ├── department created/changed ──→ ATTENDANCE (update roster)
     ├── department created/changed ──→ PAYROLL (update cost center)
     ├── department created/changed ──→ PERFORMANCE (update hierarchy)
     ├── user role changed ──────────→ RBAC permissions refresh
     └── HR settings updated ────────→ LEAVE (policy: carry-forward, max
days)
```

SHARED DATABASE MODELS REFERENCE

User.model.ts (Central — Coordinate Before Changing)

```
{
  _id: ObjectId,
  firstName: string,
  lastName: string,
  email: string,          // unique
  password: string,       // bcrypt hashed
  role: string,           // e.g., 'HR Manager', 'Employee'
  employeeId: ObjectId,   // → Employee._id (THE CRITICAL LINK)
  department: string,
  isActive: boolean,
  isDeleted: boolean,     // soft delete
  permissions: Map<string, { read, write, delete }>,  // from Role.model
  createdAt, updatedAt
}
```

Employee.model.ts (Central — Saurav Owns)

```
{
  _id: ObjectId,
  employeeCode: string,   // EMP-XXXX
  email: string,
  firstName, lastName,
  department: string,
  designation: string,
  joiningDate: Date,
  onboardingStatus: string,
  status: 'Active' | 'Inactive' | 'Terminated',
  salary: {
    basic: number,
    allowances: { hra, transport, medical, other },
    deductions: { pf, tax, insurance }
  },
  bankDetails: { accountNumber, ifsc, bankName },
  kycDetails: { aadharNumber, panNumber },
  Documents: { aadharCardPath, panCardPath, resume, ... },
```

```
  managerId: ObjectId,
  createdAt, updatedAt
}
```

Role.model.ts

```
{
  name: string,           // e.g., 'HR Manager'
  level: number,          // 1-11
  permissions: Map<string, { read: boolean, write: boolean, delete:
boolean }>,
  isSystemRole: boolean,  // system roles cannot be deleted
  createdAt, updatedAt
}
```

GIT WORKFLOW & COLLABORATION

Branch Strategy

```
main          ← Production-ready, protected. PRs only.
aditya        ← Administration, Payroll, RBAC (Aditya)
pms           ← Performance Management (Javeriya & Anjali)
recruitment   ← Recruitment & Staffing (Malhar & Pankaj)
reports       ← Reports & Analytics (Sreeshanth)
employee-center ← Employee Center (Saurav)
attendance    ← Attendance (Vishal)
```

PR Rules

1. **Never push directly to `main`** — always raise a PR
2. PR must have a description covering: what changed, which module, testing done
3. Tag the relevant person for review before merging:
   - Shared files (`User.model.ts`, `auth.middleware.ts`, `role.middleware.ts`, `server.ts`) → tag **Aditya**
   - `Employee.model.ts` → tag **Saurav**
   - Any change to `apiClient.js` → tag **all team**
4. Pull from `main` before starting new work: `git pull origin main`

Shared File Change Protocol

Before modifying any of these files, **post in team chat first**:

- `backend/server.ts` — all routes registered here; conflicts likely
- `backend/models/User.model.ts` — used by every module
- `backend/models/Employee.model.ts` — used by payroll, attendance, reports

- `backend/middleware/role.middleware.ts` — RBAC logic
- `frontend/src/services/apiClient.js` — all frontend calls go through here
- `frontend/src/components/ui/Sidebar.jsx` — nav visible to all roles

Resolving Conflicts

- **server.ts conflicts** → Aditya resolves (he maintains the route registry)
- **model conflicts** → discuss in standup; the module owner has final say on their model
- **package.json / package-lock.json conflicts** → always accept the incoming version, then run `npm install`

NOTIFICATION SYSTEM

Events & Recipients

Event	Notified Role	Channel
New user created	Admin	Email + Dashboard
Role assigned	User + Admin	Email
Payroll run initiated	Payroll Officer	Email + Dashboard
Payroll approved	Finance Manager	Email
Salary disbursed	Employee	Email + SMS
Advance request submitted	Approving Manager	Email + Dashboard
Interview scheduled	Candidate + Interviewer	Email
Offer letter issued	Candidate	Email
Attendance correction deadline	All Managers	Email (19th of month)
Integration failure	IT Admin	Email + SMS
Department transfer approved	Employee	Email

SECURITY & COMPLIANCE

Data Protection

- All salary amounts: AES-256 at rest
- Bank account numbers: last 4 digits displayed only in UI (Finance Manager sees full)
- Aadhaar/PAN: masked in UI
- API keys (integrations): never logged; shown as `***` in audit trails
- Payslips: password-protected PDF (employee date of birth as password)
- 2FA: mandatory for Finance Manager during disbursement trigger

Audit Trail

Every write operation must log to `AuditLog.ts`:

```
{
  userId, username, role,
  action,           // e.g., 'UPDATE_USER_ROLE'
  module,           // e.g., 'ADMINISTRATION'
  targetEntity, targetId,
  beforeValue, afterValue,
  ipAddress,
  status: 'SUCCESS' | 'FAILURE'
}
```

Use the `createLog()` helper — do NOT write audit entries manually.

Retention Policy

- Standard logs: 7 years
- Financial logs: 10 years
- System logs: 3 years

COMMON BUGS & THEIR FIXES

Bug	Cause	Fix
Payroll shows ₹0 for all allowances	`salaryStructure` missing `transport`, `medical`, `otherAllowance` fields	Ensure `salary.routes.ts` PUT saves all fields from `Employee.salary.allowances`
`permissions.get('payroll')` returns undefined	MongoDB `Map` serializes to plain object in JSON	Access as `permissions['payroll']` not `permissions.get('payroll')`
403 on `/api/admin/settings` for HR Manager	Role middleware not matching normalized role name	Use `authorizeByMatrix` — it normalizes role names from JWT
`res.data.data` is undefined	`apiClient` already unwraps `response.data`	Use `res.data` not `res.data.data`
Payroll lookup fails for bulk-uploaded employees	`User.employeeId` is null — Employee doc not created during bulk upload	Run `npx ts-node scripts/migrateEmployees.ts` to backfill
Attendance not locking at 25th	`isLocked` not set after payroll run initiation	`payroll.controller.ts` must call attendance lock endpoint on run initiation

Bug	Cause	Fix
Sidebar shows nothing for non-admin	Sidebar checks `userRole === 'ADMIN'` instead of reading `permissions` map	Use `canSee(moduleKey)` function already in `Sidebar.jsx`

IntersLand HRMS Master Reference v2.0 — March 2026 Prepared for all team members: Aditya, Javeriya, Anjali, Malhar, Pankaj, Sreeshanth, Saurav, Vishal